

```

class SearchPb(Object):
    def __init__(self, s_in):
        self.s = s_in
    def acciones(self, s):
        raise NotImplementedError('A desarrollar')
    def sucesor(self, s, q):
        raise NotImplementedError('A desarrollar')
    def terminal(self, s):
        raise Not...

```

± | |

$S = (s_1, \dots, s_5)$
 $s_i \in \{'A', 'B', 'C'\}$

```

class TorresHanoi(SearchPb):

```

```

    def __init__(self, s0 = 5 * ['A'], sf = 5 * ['C']):
        self.s0 = s0
        self.sf = sf
        self.acciones = {'AB', 'AC', 'BA', 'BC', 'CA', 'CB'}

```

```

    def acciones(self, s):
        if any(s_i not in ['A', 'B', 'C'] for s_i in s):
            raise ValueError('Estado(s) imposible')
        return [q in self.acciones if q[0] in s and
                (q[1] not in s or s.index(q[0]) < s.index(q[1]))]

```

```

    def sucesor(self, s, q):
        costo_local = 1
        if q not in self.acciones(s):
            raise ValueError('—')
        s_next = s[:]
        s_next[s.index(q[0])] = q[1]
        return s_next, costo_local
    def terminal(self, s):
        return s == self.sf

```

Simulación aleatoria

```

torres = TorresHanoi(s0, sf)
historial = []
s = s0[:]
for _ in range(5000):
    acciones = torres.acciones(s)
    q = random.choice(acciones)
    s_n, c_l = torres.sucesor(s, q)
    s = s_n[:]
    c += c_l
    historial.append(q)
    if torres.terminal(s):
        break
else:
    print('se resolvió o lo mismo en {costo} pasos')

```